

Chunking Strategies, LangChain Intro & RAG Categories

Fixed Size • Hierarchical • Semantic • Docling • LangChain • Hybrid RAG • Graph RAG • SQL RAG
• HNSW

1. Topic Overview

Session 4 picked up right where the RAG revision left off and moved from theory into two practical skills: how to split real documents into good chunks, and how to rebuild the previous day's RAG pipeline using LangChain instead of raw Python. The second half of the class then zoomed out to a bigger question: once you have a working RAG pipeline, which of the many flavors of RAG should you actually reach for. The session closed with a bonus walkthrough of HNSW, the algorithm that lets vector databases such as Qdrant search millions of vectors quickly.

Area	What was covered
RAG revision	Indexing and retrieval phases, and fixed size chunking with overlap, as a recap from the previous session
Chunking strategies	Fixed size, hierarchical, semantic, sentence window, layout aware, custom, and hybrid chunking, plus when to reach for each one
Docling	A Python library that turns messy PDFs into structured, chunkable text
Hands on: doc parsing	Parsing a PDF with Docling and applying hierarchical chunking, using Qdrant and Sentence Transformers
LangChain introduction	Where LangChain fits in the AI stack, its core building blocks, and why teams reach for it
Hands on: RAG in LangChain	Rebuilding the RAG pipeline with a Qdrant vector store, a retriever, and a prompt template
RAG categories	Vector (naive), keyword, hybrid, graph, SQL, and reasoning based RAG, and where each one is actually used
HNSW (bonus)	How hierarchical navigable small worlds power fast vector search, revisited from the previous session

Industry Context

Hybrid RAG (vector search plus keyword search) and Text to SQL RAG are the two categories that, according to Sid, solve 80 to 90 percent of real production use cases today. Graph RAG and reasoning based RAG (PageIndex) exist for narrower, more specialized problems and show up far less often in production.

2. Key Concepts

2.1 RAG in Two Phases: A Quick Revision

RAG lets an LLM answer questions on information it was never trained on, such as a company's internal HR policies. When the source knowledge is too large to fit into a single prompt, you divide it into chunks, create embeddings of those chunks, and store them in a vector database. That first phase is called indexing. The second phase, retrieval, takes a user's question, embeds it the same way, searches the vector database for the closest chunks, and hands those chunks to the LLM as context so it can generate an answer. Everything covered in today's session builds on these same two phases, just with better chunking and with LangChain doing most of the plumbing.

2.2 Chunking Strategies

Fixed size chunking splits a document into equal sized pieces, for example 500 characters or 500 tokens at a time, and usually adds a chunk overlap of 10 to 20 percent so that consecutive chunks share a little text. This keeps some continuity between chunks, the same way you might briefly recap last week's session before starting a new one, so the model is not left reading a chunk with no sense of what came before it.

Hierarchical chunking carries a document's headers and subheaders into every chunk. For a policy document with a main header, a section header, and a subsection header, each chunk includes all three levels along with its own content, so a chunk about probation policy still says which section and which overall document it belongs to. Semantic chunking instead splits based on where the meaning actually changes, for example separating the preparation, cooking, and garnishing steps of a recipe into three separate chunks because each is a distinct logical unit, even if the sections are different lengths. If one semantic section grows too large, you can nest fixed size chunking inside it.

Sentence window chunking sizes a chunk by a number of sentences rather than characters or headers, which works fine for plain articles and knowledge base pages but breaks down as soon as a document contains tables or code. Layout aware chunking extends hierarchical chunking to also understand a document's visual layout, such as multiple columns, tables, and diagrams, using a vision language model to read the page the way a person would rather than left to right in a straight line. It is powerful for scanned reports but is overkill for plain text. Custom chunking is whatever bespoke logic a team designs for its own documents. AtliQ, for example, built a custom pipeline that blends semantic chunking with fixed size chunking for its own use case. Hybrid chunking is simply the umbrella term for combining more than one of these strategies in the same pipeline, chosen based on the specific use case.

Key Insight

Do not confuse hybrid chunking with hybrid RAG. Hybrid chunking means combining more than one chunking strategy. Hybrid RAG means combining vector search with keyword search. They solve different problems even though they share a name, and it is an easy mix up to make.

2.3 Choosing the Right Chunking Strategy

Strategy	Best for	Watch out for
Fixed size (with overlap)	A quick baseline for almost any text	Can split a thought, a sentence, or a table in the middle
Hierarchical	Documents with clear headers and subheaders, such as policies and manuals	Fails on flat, unstructured text with no real structure
Semantic	Long passages where the topic drifts, such as research papers	Resource intensive, since it needs an embedding model to detect topic shifts before it can chunk
Sentence window	Plain articles and knowledge base style pages	Breaks down once tables or code enter the document
Layout aware	Scanned PDFs with columns, tables, and diagrams	Overkill for a pure text document
Custom	A specific business need, like AtliQ's HR policy pipeline	Needs an engineer to design, test, and maintain the logic
Hybrid (chunking)	Real world document sets that mix several formats	Adds complexity, since more than one strategy has to be maintained

Key Insight

There is no single right chunk overlap. The common starting point shared in the session was 15 to 20 percent of the chunk size, with chunk sizes often starting around 500 characters or tokens, give or take 100. From there, treat it as an experiment: start with a reasonable baseline, evaluate retrieval quality on real test questions, and adjust.

2.4 Docling: Parsing Messy Documents

Before you can chunk a document, you first have to read it, and a PDF is not a plain text file. It can carry tables, images, scanned pages, and formulas, and it may need OCR before any of that content is usable. Docling is an open source Python library that converts these messy documents into clean, structured data, so that downstream chunking and RAG work on something predictable. Sid pointed to two comparable tools,

Unstructured (also open source) and Azure AI Document Intelligence (a paid cloud service), and explained that AtliQ chose Docling to stay fully open source. Asked whether ChatGPT itself uses Docling when a document is uploaded, Sid said only OpenAI's own engineers would know for certain, but some kind of document parsing step is almost certainly running before the content ever reaches the LLM, whether that is Docling, Unstructured, or Microsoft's own tooling.

2.5 LangChain: What It Is and Why It Exists

To place LangChain in context, Sid walked through a short history of AI progress: the deep learning wave that began with AlexNet, followed by speech recognition and medical imaging, then generative AI in 2022 when ChatGPT launched, and now the current Agentic AI phase, with physical AI (robots, self driving cars) still ahead. He offered a simple analogy: the LLM is the brain of an AI application, and the application itself is the body, the hands, eyes, and other senses built around that brain. Your job as an AI engineer is to build that body. LangChain is a framework that makes building that body dramatically easier: the same pipeline that takes 100 lines of plain Python can often be written in about 20 lines with LangChain, because it abstracts away a lot of repetitive boilerplate.

LangChain's core building blocks are document loaders and utilities, prompt handling, LLM wrappers that support most major providers, chains, agents, and tools. In the chat, the team and cohort surfaced two concrete production reasons to use it: first, it lets you swap a vector database, embedding model, or LLM provider by changing a library rather than rewriting your pipeline, which avoids vendor lock in. Second, it comes with utility functions that speed up development. The trade off is a small loss of manual control, for example some configuration is needed to fully control metadata, though LangChain does support storing metadata, clarifying a question that came up in chat.

Key Insight

LangChain's biggest production advantage, as discussed in the session, is not that it lets you do something new. It is that it lets you swap out a vector database, an embedding model, or an LLM provider by changing a library instead of rewriting your pipeline.

2.6 RAG Categories

Category	How it works	Best for	Key tools
Vector RAG (naive)	Embeds chunks and retrieves the top K by cosine similarity	Semantic, meaning based questions	Qdrant, ChromaDB, Milvus
Keyword RAG (vector less)	Matches on keywords using BM25 or TF IDF, with no embeddings at all	Code, jargon, IDs, citations, and precise keyword queries	Elasticsearch, Apache Solr, Apache Lucene
Hybrid RAG	Runs vector search and keyword search in parallel and merges the results	Most production systems, according to Sid	Elasticsearch or Solr plus any vector database, or Qdrant's built in keyword support
Graph RAG	Builds a knowledge graph of entities and relationships, then traverses it to answer the question	Questions about how entities connect, for example which companies a person founded	Graph databases and knowledge graph tooling
SQL RAG (Text to SQL)	An LLM writes a SQL query from the question, the query runs against a real database, and the result is turned back into a natural language answer	Structured business data already sitting in a SQL database	Any relational database plus an LLM
Reasoning based RAG (PageIndex)	An LLM builds a hierarchical index of a long document's sections, then traverses just that index to fetch the right paragraph	Very long documents, such as legal contracts, though still experimental	PageIndex, an open source library still in early stages

Industry Context

Only Hybrid RAG and SQL RAG get hands on treatment next week, because together they cover most real deployments. Graph RAG content is being added to the recorded bootcamp by September 5, and reasoning based RAG is still too new and too expensive to see much production use. A related idea, agentic RAG, came up in chat as well: Sid described it simply as handing the retriever to the LLM as one of its own tools, rather than calling it yourself inside a fixed pipeline, and confirmed it will be covered later in the cohort.

2.7 Bonus: HNSW, the Engine Behind Fast Vector Search

To close the class, Sid revisited HNSW, short for hierarchical navigable small worlds, the algorithm that Qdrant and most modern vector databases use internally to make similarity search fast. Comparing a query vector to

every single stored vector one by one would take time proportional to the number of vectors, which does not scale. HNSW instead arranges vectors into several stacked layers of interconnected small worlds. As each new chunk is added, it is placed based on its cosine similarity to nearby points, so similar chunks end up close together and dissimilar ones end up farther apart. A search always starts at the top layer and moves downward, narrowing in on the closest matches at the lowest layer, the way you might travel by flight for a long distance, then switch to a bus or train, then finally a car for the last short stretch. There is no single root node and no leaf node, unlike a tree, which is why it is called a graph of small worlds rather than a hierarchy in the strict sense.

Key Insight

HNSW is not a tree. There is no single root node. Think of it as several layers of connected small worlds: a search always travels down to the lowest layer, then ranks the points it crossed by cosine similarity to pick the closest matches. This is a common interview topic, so it is worth revisiting the recording if any part of it felt fuzzy.

3. Hands On Walkthroughs

3.1 Doc Parsing and Chunking with Docling

The first notebook worked entirely without LangChain, by design, so the cohort could see the underlying pieces before LangChain hides them. It imported a document converter, a hierarchical chunker, a result post processor, a Sentence Transformer model, and a Qdrant client. Instead of the plain text file used the day before, this notebook worked with a real PDF version of the same AtliQ HR policy document. Because the chunking strategy here follows the document's own headings rather than a token count, no separate tokenizer was needed at the chunking step, unlike the fixed size chunking exercise from the previous day, which did use one.

Field	What it holds
headings	The path of headers and subheaders a chunk belongs to, for example AtliqAI HR Policies, Employment and Onboarding, Probation Period
content	The actual paragraph text captured in that chunk
chunk_text	The headings joined into a breadcrumb, followed by the content, which is what actually gets embedded and stored

The breadcrumb (the joined heading path placed ahead of the content) matters at retrieval time: it means a chunk about probation period still carries the context of which section and which overall policy document it came from, so the cosine similarity match and the final answer both stay grounded in the right part of the document, not just the isolated sentence.

3.2 Building RAG in LangChain

The second notebook rebuilt the same pipeline using LangChain. A Qdrant vector store was created directly from the documents in a single call, which handled starting the embedding model, creating the collection, and ingesting every chunk automatically, replacing the manual embedding and ingestion code written the day before. Calling as retriever on that vector store, with a search argument for how many chunks to fetch (kept at 5 for this exercise), produced a retriever object, and retriever.invoke on a query returned the relevant chunks directly.

From there, a chat prompt template was built from a short system message (answer only using the context below, cite section names, say you do not know if unsure) and a human message containing the context and question as variables. The full pipeline was then just a few lines: retrieve chunks for the query, concatenate their text into one context, populate the prompt template with that context and the original question, and pass the finished prompt to the LLM. Run against three sample questions, it correctly answered that casual leave entitlement is 12 days per calendar year, that the notice period for Band 4 employees is 60 days, and that the probation period is 6 months, matching the manual pipeline from the day before with far less code.

4. Session Output and Next Steps

Item	Detail
This week's exercise	Use LangChain's Docling integration for document ingestion, then rewrite the entire RAG pipeline built so far using LangChain end to end
Next week: hands on	Hybrid RAG and Text to SQL RAG, the two categories used in most production systems
Recorded bootcamp	Graph RAG content is being added to the recorded bootcamp by September 5, for anyone who wants to go deeper
Formal assignment	Will be issued at the next checkpoint, per Sid's note in the session

5. Quick Revision Summary

Term	Meaning
Indexing	Turning documents into chunks, embedding them, and storing them in a vector database
Retrieval	Finding the most relevant chunks for a query and handing them to the LLM as context
Chunk overlap	Repeating a small amount of text between consecutive chunks so context is not lost
Hierarchical chunking	Chunking that carries a document's headers and subheaders into every chunk
Semantic chunking	Chunking based on where the topic or meaning actually changes
Docling	An open source library that converts messy PDFs into structured, chunkable text
LangChain	A framework that abstracts away the boilerplate of building LLM applications
Hybrid RAG	Combining vector (semantic) search with keyword search
Graph RAG	Answering questions by traversing a knowledge graph of entities and relationships
SQL RAG (Text to SQL)	Having an LLM write and run a SQL query, then explain the result in plain language
Reasoning based RAG (PageIndex)	Building a hierarchical index of a long document and traversing just that index
HNSW	The layered small world algorithm vector databases use to search millions of vectors quickly